# Validation of an Edge-Fog-Cloud Scheduling Testbed

**Gleb Mozhaiskii · Research note · October 2026**

Prepared for discussion with José Pascual and Unai Garciarena (UPV/EHU).

## Abstract

I evaluated whether an application-level scheduling testbed could be reconstructed and execute the same workload correctly across three fresh Linux VM sessions. Each session contained a Gateway orchestrator and three CPU-limited workers connected through MADT, with global, sender and worker queues. The prospective protocol fixed the runtime baseline, 12 scheduling scenarios, validation criteria and stopping rules before collection. All three sessions completed: 2,844 worker outputs were reference-checked, including 432 scored tasks. Of these, 431 met the one-second deadline measured from planned arrival; one correct result was late. Network qualification, evidence preservation and cleanup passed their separate checks. These results establish a bounded same-host correctness and repeatability screen. Placement, latency and decision-cost observations are reported separately from that validation objective.

## 1. Research Objective

The predefined question was whether the existing orchestrator could reconstruct its runtime and correctly execute and account for the original pilot across **three fresh VM sessions on the same host**. A separate question concerned reproduction of the configured network conditions within the original numerical tolerances.

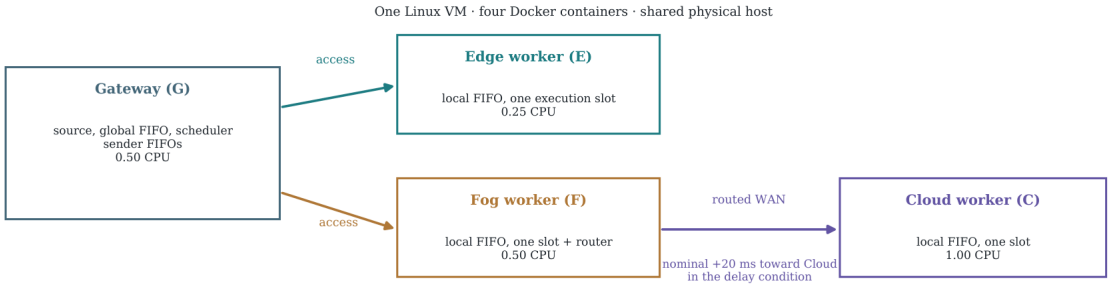
The application screen required three complete sessions. Completion required all expected task outcomes and reference outputs, consistent assignments and queue transitions, one execution slot per worker, finite resource accounting, valid clocks and provenance, intact evidence export, and verified cleanup. Network qualification was assessed independently. Strict performance eligibility additionally required both network qualifications to pass and the preserved evidence and cleanup to be verified. A correct result that missed its deadline remained a late result; lateness alone did not fail application correctness.

RR, Compute-ECT and Network-ECT exercised the instrument under different placement decisions. Their performance ordering was not a validation criterion. The policy comparisons in Section 6 are exploratory observations from the completed cohort.

Wang et al. [1] provide methodological context: compare controllers under common conditions and report decision overhead alongside service-level outcomes. Their Continuous Multi-Mode Scheduling benchmark selects execution modes as well as destinations for video inference. This testbed instead assigns fixed CPU tasks to workers and checks their results.

## 2. Testbed Architecture

Each session used a fresh ARM64 Lima Linux VM with 4 vCPU, 4 GiB RAM and a 20 GiB sparse disk. The three sessions ran sequentially on the same Apple Silicon Mac. MADT [2] deployed the four-container topology below. G/E/F/C CPU quotas were 0.50/0.25/0.50/1.00 CPU, with a 100 ms period and a 256 MiB memory limit per container.




System topology

Gateway G owns the global FIFO, placement controller and one sender FIFO per worker. Each sender serializes uploads until input-admission acknowledgement. Workers E, F and C each have a local FIFO and one non-preemptive execution slot; execution can overlap the next input transfer. G reaches C through F, while E is directly reachable on the access network. F therefore shares its quota between routing and application work.

Each worker permits at most 32 unresolved assignments and 64 MiB of application reservations. A task reserves its input size plus 1 MiB; capacity is released after a validated result. These bounds are distinct from the container memory limit. Worker state is reported every 100 ms, and placement uses an immutable snapshot of controller-known events and state.

The workload is deterministic PBKDF2-HMAC-SHA256 computation over a digest of the transferred input. Two input sizes (64 KiB and 1 MiB) and two work levels (200,000 and 800,000 iterations) define four task classes. G transfers the complete input and independently checks the returned result vector. Task, assignment and output identities are checked within each execution group.

## 3. Scheduling Policies

All policies use the same eligibility rule: the worker must be online, below its unresolved-assignment limit and able to reserve the input plus 1 MiB. A busy CPU does not by itself make a worker ineligible; work can wait in its queues. RR advances through E, F and C, skipping ineligible workers. The ECT policies select the eligible worker with the lowest predicted response-completion time; ties use the same E/F/C order. RR supplies a profile-free baseline. The two ECT variants keep service and queue accounting fixed while changing whether transport estimates enter the score.

For task $k$ and worker $j$, let $s_j(k)$ be calibrated execution wall time. Let $u_j(b)$, $a_j(b)$ and $v_j(b)$ be the calibrated input-delivery, subsequent acknowledgement and result-return durations for input size $b$. Service estimates are ceiling-rounded arithmetic means of three unshaped calibration

samples per worker/task class. Transport estimates use all six samples per worker/input size in each network condition. The profiles are frozen before comparison and shared across policies; transport profiles are condition-specific.

At decision time $t$, the residual estimate for a reported executing task with start time $x$ is

$$R_j(t) = \max\{0,\ s_j(k_{\text{run}}) - (t - x)\}.$$

It is zero when no task is reported executing. An exhausted estimate contributes zero residual, but the assignment remains unresolved until its result is verified.

**Compute-ECT.** If $W_j$ contains assigned tasks still waiting for transfer or execution, the candidate $i$ is scored as

$$\widehat{F}_j^{\mathrm{C}}(i) = t + R_j(t) + \sum_{k \in W_j} s_j(k) + s_j(i).$$

This is the implemented predictor with input, acknowledgement and result-transfer estimates set to zero. Tasks already executing, returning results or completed are excluded from $W_j$.

**Network-ECT.** The predictor retains sender serialization and overlap between input delivery and execution. Initialize the execution frontier $C_j = t + R_j(t)$ and upload frontier $U_j = t$. If an upload started at $e$ and its acknowledgement is outstanding, initialize $U_j = \max\{t, e + u_j(b) + a_j(b)\}$ instead. This also applies when its computation finishes before the acknowledgement arrives.

Traverse the remaining assigned tasks in assignment order and append the candidate. Skip tasks already executing, returning or completed. For each remaining task $k$, use the first applicable case below. A task already in the worker FIFO is ready at $t$ even while its sender ACK remains outstanding:

$$A_{jk} = \begin{cases} t & \text{already in worker FIFO,} \\ \max\{t, e_k + u_j(b_k)\} & \text{active upload,} \\ U_j + u_j(b_k) & \text{upload not yet started.} \end{cases}$$

Only a not-yet-started upload advances $U_j \leftarrow A_{jk} + a_j(b_k)$; the active upload is already represented in the initial frontier. After each task, update

$$C_j \leftarrow \max\{C_j, A_{jk}\} + s_j(k), \qquad \widehat{F}_j^{\mathrm{N}}(i) = C_j + v_j(b_i)$$

where the final score uses $C_j$ after appending the candidate. Result return contributes to the candidate's response estimate without occupying its worker's CPU slot. These recurrences describe the frozen implementation; they use no future completion observations.

## 4. Experimental Protocol

The prospective cohort comprised three sequential fresh VM sessions, Run 1-3, attempted once each without replacement or tuning. Each reconstructed the pinned runtime and used the same task trace, policy definitions, quotas and hash-derived scenario order. The baseline included Lima 2.2.1, Docker 28.5.2, containerd 1.7.29 and the pinned MADT revision and network-control repair. Full source identities, dependency pins and execution commands accompany the public artifact.

Each session crossed three policies, two arrival patterns and two network conditions: **12 scenarios, each with 12 scored offers over six seconds**. The steady trace releases one task every 0.5 s. The burst trace combines a one-task/s stream with three additional tasks near 2 s and three near 4 s. Task classes and release times are identical across policies and sessions.

The scheduling conditions were unshaped networking and a nominal 20 ms one-way delay on Cloud ingress from Fog. The separate 10 Mbps treatment qualified network control; it was not a scored scheduling condition. Each VM performed 72 calibration tasks, 12 queue-check tasks, 720 common warmup tasks and 144 scored tasks: 948 executions in total.

**Application criteria.** All 15 execution groups, including the 12 scored scenarios, had to complete with 948 independently checked worker outputs and all 144 scored outcomes retained. The audit checked task conservation, assignment/output identities, FIFO and single-slot constraints, finite credits and reservations, queue/execution overlap, causal timestamps, resource limits and source/runtime provenance. Missing, invalid or incompletely audited application evidence required a stop.

**Network criteria.** Both health and application-quota qualification measured baseline, impaired and restored states. For both probe directions, the median RTT increment had to be 15-25 ms and the restored median within 2 ms of baseline. The unaffected access-path medians could span at most 2 ms. The rate treatment required 7-11 Mbps, with baseline and restoration at least 20 Mbps and restoration at least 75% of baseline. Ordinary measurements required 20/20 ping delivery and valid TCP delivery. Route, interface, qdisc and numerical actuation readbacks, partition/recovery and unaffected-path checks were also mandatory. A valid numerical deviation was recorded separately from application correctness and excluded the session from strict performance comparison; it did not become a relaxed network pass.

**Measurement and preservation.** A scored task meets its deadline when $z_i - a_i^* \leq 1$ s, from planned release $a_i^*$ to verified response $z_i$. The record separates release lag, global waiting, observation, decision, commitment, sender waiting, input delivery, worker waiting, execution and result return. Decision wall time measures the timed policy call; thread CPU time is recorded separately. Containers share a checked Linux monotonic-clock domain. Export integrity and actual session cleanup were verified before publication eligibility was finalized.

## 5. Validation Results

All three sessions passed the application screen and both network qualifications. Evidence export, preservation and cleanup passed their recorded audits. All 432 scored offers completed correctly: none was rejected, failed, unresolved or unreleased. The three sessions produced 2,844 reference-checked worker outputs including preparation.

| Session | Application | Correct outputs | Scored on time |
|---|---|---|---|
| Run 1 | Complete | 948 | 144/144 |
| Run 2 | Complete | 948 | 144/144 |
| Run 3 | Complete | 948 | 143/144 |

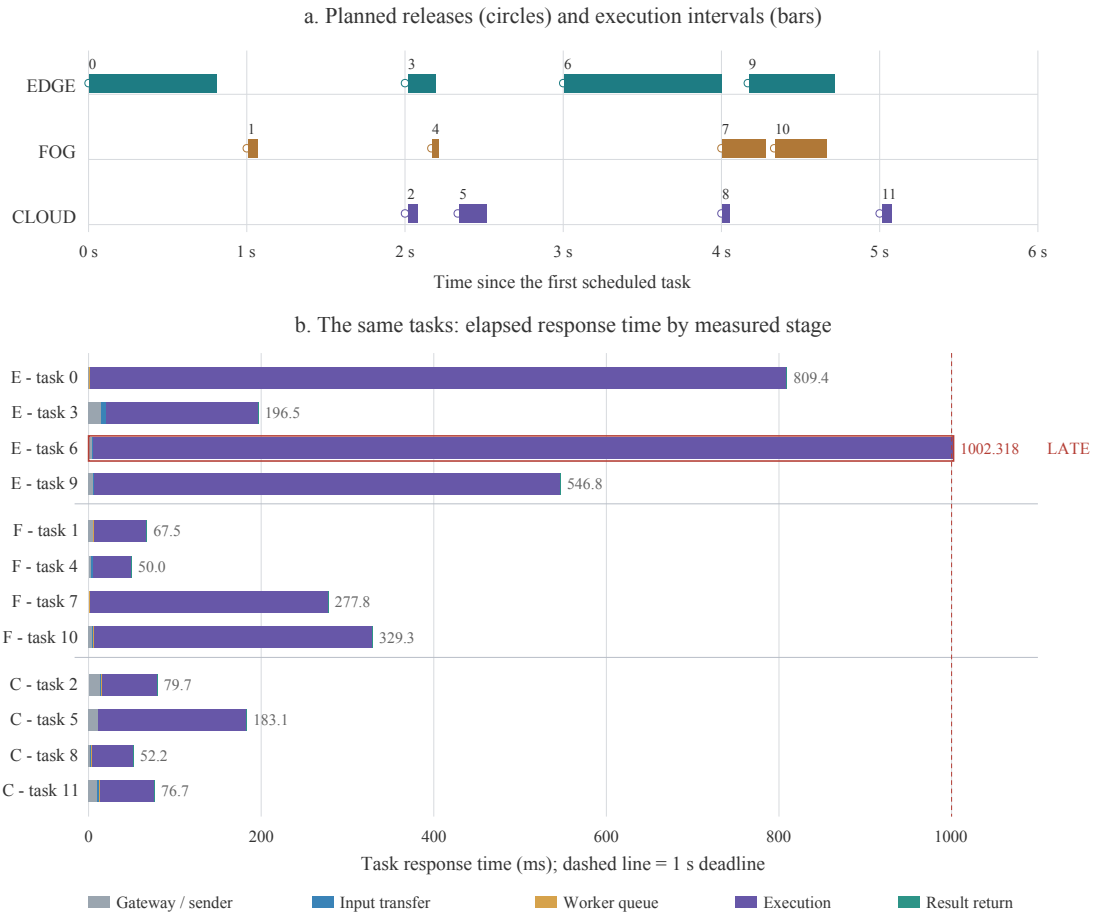
The Gateway recorded 948 assignments per session, including 144 scored decisions. Worker totals below include calibration, queue checks and warmup; parentheses contain scored outputs. Every worker produced verified results in each session.

| Session | G all / scored | Edge | Fog | Cloud |
|---|---|---|---|---|
| Run 1 | 948 / 144 | 284 (16) | 294 (26) | 370 (102) |
| Run 2 | 948 / 144 | 284 (16) | 294 (26) | 370 (102) |
| Run 3 | 948 / 144 | 284 (16) | 298 (30) | 366 (98) |

The application-quota network qualification produced the following RTT increments and limited receive rates. Forward and reverse denote probe initiation directions; these are RTT increments, not direct one-way-delay estimates. All values lie within the original 15-25 ms and 7-11 Mbps bands; restoration, delivery and control-path checks also passed.

| Session | Forward delta (ms) | Reverse delta (ms) | Limited rate (Mbps) |
|---|---|---|---|
| Run 1 | 22.139 | 20.779 | 9.072 |
| Run 2 | 21.223 | 21.570 | 9.320 |
| Run 3 | 22.045 | 21.967 | 8.357 |

Of the 432 scored tasks, **431 met the one-second planned-arrival deadline**. The remaining task was an Edge assignment in Run 3, RR/burst/unshaped. Its correct response arrived after 1002.317940 ms, exceeding the deadline by 2.317940 ms. Release lag was 4.019139 ms; measuring from actual release would yield 998.298801 ms and change its classification. The predefined planned-arrival result is retained.

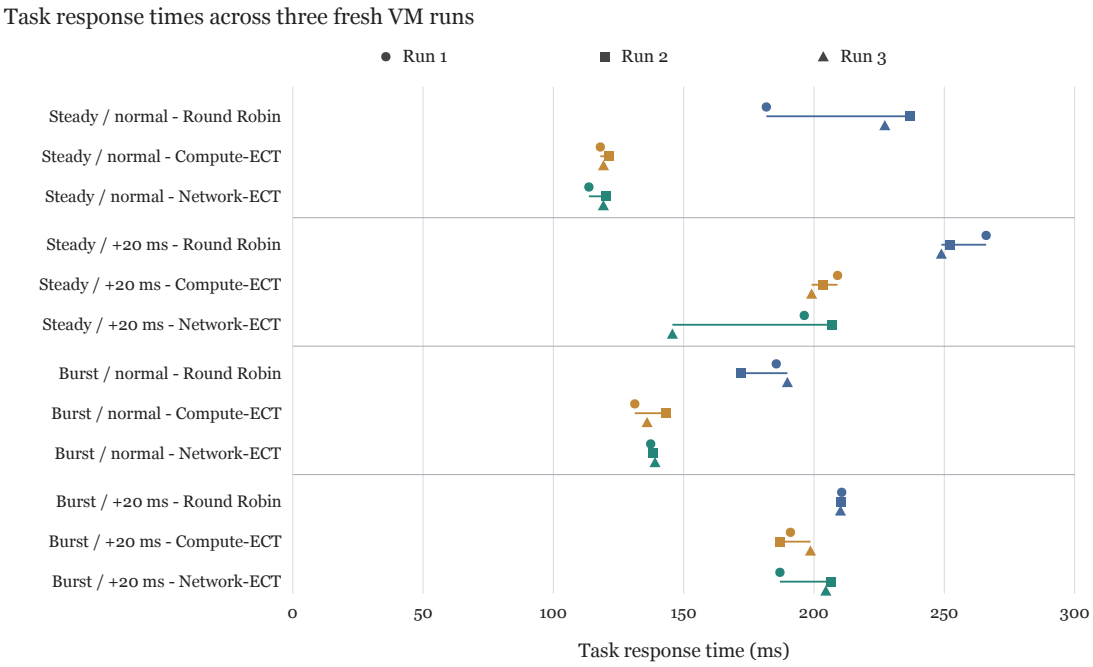



Run 3 task timeline for Round Robin with burst arrivals; the late task is retained

## 6. Exploratory Observations

The **VM session is the replication unit: N=3 on one host**. Each run cell below gives median response time in milliseconds and, beneath it, assignments to Edge / Fog / Cloud, out of 12 tasks. 'On time' is a descriptive total over all 36 offers in that row. The rows and tasks are nested observations, not additional independent replications. All 12 scenario combinations are shown.

| Arrival | Network | Scheduler | Run 1 | Run 2 | Run 3 | On time |
|---|---|---|---|---|---|---|
| Steady | Normal | Round Robin | 181.7<br>4 / 4 / 4 | 236.7<br>4 / 4 / 4 | 227.2<br>4 / 4 / 4 | 36/36 |
| Steady | Normal | Compute-ECT | 118.0<br>0 / 0 / 12 | 121.2<br>0 / 0 / 12 | 119.2<br>0 / 0 / 12 | 36/36 |
| Steady | Normal | Network-ECT | 113.6<br>0 / 0 / 12 | 120.1<br>0 / 0 / 12 | 119.2<br>0 / 0 / 12 | 36/36 |
| Steady | +20 ms | Round Robin | 266.1<br>4 / 4 / 4 | 252.4<br>4 / 4 / 4 | 248.9<br>4 / 4 / 4 | 36/36 |
| Steady | +20 ms | Compute-ECT | 209.0<br>0 / 0 / 12 | 203.5<br>0 / 0 / 12 | 199.1<br>0 / 0 / 12 | 36/36 |
| Steady | +20 ms | Network-ECT | 196.3<br>0 / 3 / 9 | 207.0<br>0 / 3 / 9 | 145.7<br>0 / 6 / 6 | 36/36 |
| Burst | Normal | Round Robin | 185.5<br>4 / 4 / 4 | 171.9<br>4 / 4 / 4 | 189.8<br>4 / 4 / 4 | 35/36 |
| Burst | Normal | Compute-ECT | 131.2<br>0 / 1 / 11 | 143.3<br>0 / 1 / 11 | 136.0<br>0 / 1 / 11 | 36/36 |
| Burst | Normal | Network-ECT | 137.3<br>0 / 1 / 11 | 138.3<br>0 / 1 / 11 | 139.0<br>0 / 1 / 11 | 36/36 |
| Burst | +20 ms | Round Robin | 210.6<br>4 / 4 / 4 | 210.4<br>4 / 4 / 4 | 210.2<br>4 / 4 / 4 | 36/36 |
| Burst | +20 ms | Compute-ECT | 191.0<br>0 / 1 / 11 | 186.8<br>0 / 1 / 11 | 198.7<br>0 / 1 / 11 | 36/36 |
| Burst | +20 ms | Network-ECT | 186.9<br>0 / 4 / 8 | 206.5<br>0 / 4 / 8 | 204.5<br>0 / 5 / 7 | 36/36 |



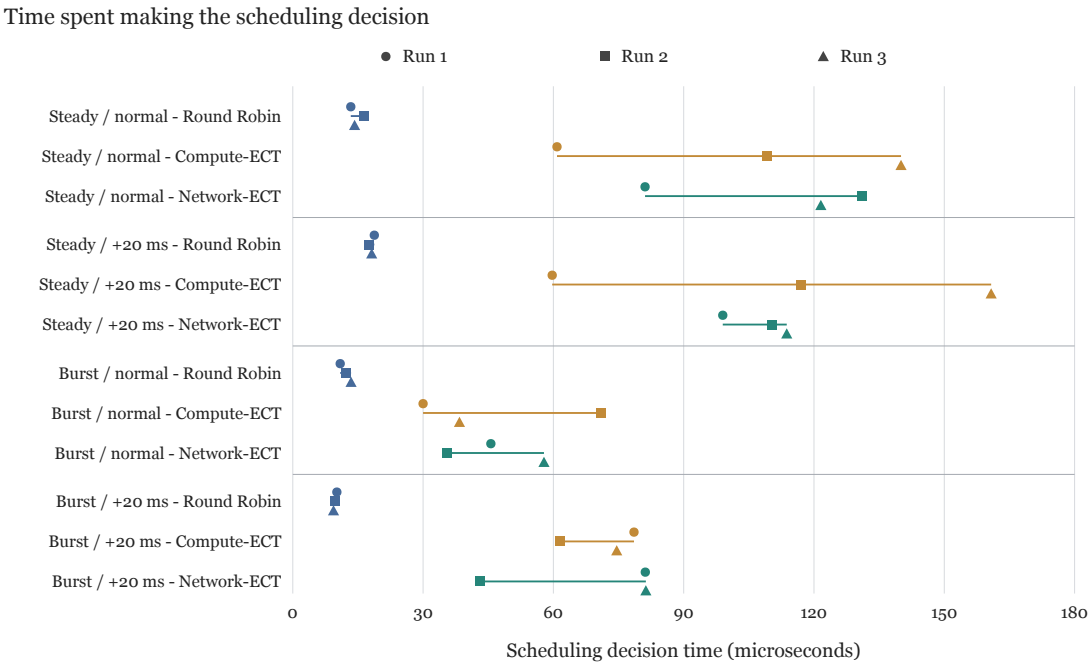

Median task response time in each fresh VM run

Each symbol is one session's scenario median. The horizontal segment spans the observed minimum and maximum of the three medians; it is not a confidence interval.

Both ECT response medians were lower than the matched RR median in every observed session/context pair. Across fixed scenarios, the relative range $(\max - \min)/\text{median}$ was 0.219%-31.267%.

**Placement.** RR assigned 4/4/4 in every scenario. Neither ECT policy selected Edge in this cohort. With burst arrivals and added delay, Compute-ECT assigned one task to Fog per run; Network-ECT assigned 4, 4 and 5. Under steady arrivals and added delay, Network-ECT assigned 3, 3 and 6 tasks to Fog. Its corresponding latency medians were 196.273, 207.037 and 145.668 ms, the largest inter-session spread. The placement change and lower Run 3 median are associated observations; calibration and execution timing also varied between sessions.

**Decision cost and offered load.** Decision wall medians across the 36 scenario/session points ranged from 9.375 to 160.792 microseconds; CPU medians ranged from 6.459 to 146.083 microseconds. In all 24 matched ECT-minus-RR comparisons, both decision wall and CPU medians were higher for ECT. Decision cost remains included in measured task response.




Scheduling decision wall time in all three sessions; each point is one scenario median

Every scenario completed its 12 scored tasks within the six-second window: observed window throughput was 2 tasks/s. On-time goodput was also 2 tasks/s except for the late-task scenario, where it was 11/6 tasks/s. This offered load does not identify maximum capacity or overload behavior.

## 7. Threats to Validity

**Replication and scope.** Three fresh VMs shared one physical host and operator. They tested runtime reconstruction and repeated execution with one recipe, a fixed seed and a fixed scenario order. The data do not establish portability, long-term reliability or a general policy ranking. CPU quotas and emulated links represent logical tiers.

**Timing and attribution.** CPU contention, quota replenishment, virtualization, network emulation and measurement activity all contribute to wall time. They were not varied independently, so the deadline miss and inter-session spread cannot be assigned to a single cause. The range of three medians is descriptive uncertainty, with too little replication for broad statistical conclusions. Decision wall time includes elapsed scheduling and instrumentation effects; it is not total control-plane cost.

**Workload and models.** Twelve scored tasks per scenario and four deterministic CPU classes provide a correctness workload. They do not cover stateful services, application dependency graphs, migration, recovery of interrupted work or sustained saturation. Service profiles come from unshaped calibration; transport profiles are condition-specific and fixed. Model error under changing load or stale state was not experimentally isolated. Fog also couples forwarding and execution.

**Reconstruction.** Source, versions and package manifests were pinned and checked, but the environment was not rebuilt from a hermetic package mirror. Future dependency availability remains an operational risk. All three collected sessions are included; preparation and warmup outputs are distinguished from scored outcomes.

## 8. Conclusion and Research Directions

The predefined same-host application screen passed in three fresh VM sessions, with complete task accounting and verified outputs on every worker. The configured network treatments passed their separate qualifications, and preserved evidence and cleanup supported final comparison eligibility. The scored outcome was 431 on-time results and one late correct result under the original planned-arrival deadline.

The next scientific question is how placement quality changes when queue and network observations become stale, and what information-update cost is justified by the resulting service-level benefit. The present implementation provides a measured baseline for designing that study. Extending the question to temporary worker loss or recovery would require an explicit failure and task-state model; the current cohort supplies no recovery evidence.

## References


1. Z. Wang, B. Sedlak, J. L. Herrera, and S. Dustdar. 'Fair Comparison of Scheduling Algorithms on Heterogeneous Edge Clusters: A Continuous Adaptive Benchmark.' IEEE ICDCSW, pp. 232-237, 2026. DOI: 10.1109/ICDCSW72724.2026.00054. Open preprint: arXiv:2606.12343.

2. MADT project. 'Distributed Application Modelling System.' Source repository, accessed 23 October 2026.